UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE D'AGADIR
المدرسة العليا الخاصة للهندسة بأكادير

Système d'exploitation Linux

Pr. Salwa BELAQZIZ
s.belaqziz@uiz.ac.ma

Année universitaire 2026-2027

Notions de base

Système d'exploitation, historique d'Unix et de Linux, architecture

Le système d'exploitation fait l'intermédiaire entre les applications et le matériel

Utilisateurs

Applications : navigateur, WhatsApp, appareil photo

Programmes d'exploitation : éditeur, gestionnaire d'impression

Système d'exploitation (OS)

Matériel : processeur, mémoire, disque, clavier

Définition

Un **système d'exploitation** (SE, en anglais operating system, OS) est un ensemble de programmes qui **pilote les ressources matérielles** et **fournit des services** aux applications et aux utilisateurs.

Pourquoi une couche intermédiaire ?

Pour fonctionner, un programme échange avec le processeur, la mémoire et les périphériques. L'OS assure ces échanges à sa place.

Exemples d'OS : Windows, macOS, Unix, Linux, Android.

Sous Unix/Linux, l'OS offre ses services aux programmes sous forme d'**appels système**, écrits en langage C.

Un OS joue trois rôles : traducteur, chef d'orchestre et agent de sécurité



DÉFINITION 1

Traducteur

Il traduit les demandes des programmes en opérations sur le matériel.

Ex. : « enregistrer le fichier » devient une suite d'écritures sur le disque.



DÉFINITION 2

Chef d'orchestre

Il organise et répartit l'accès des programmes aux ressources, et fixe leur ordre d'exécution.

Ex. : quatre programmes demandent le processeur ; l'OS décide qui passe et quand.



DÉFINITION 3

Agent de sécurité

Il interdit aux programmes l'accès direct au matériel, sauf autorisation, et protège l'intégrité du système.

Ex. : une application ne peut pas activer la caméra sans permission.

L'OS assure cinq tâches de gestion pour le compte des programmes

01

Processus

Partage le processeur entre les programmes en cours d'exécution.

02

Mémoire

Attribue la mémoire vive à chaque programme et la récupère quand il se termine.

03

Fichiers

Gère la lecture et l'écriture sur le disque, ainsi que les droits d'accès aux fichiers.

04

Entrées/sorties

Contrôle les périphériques via les pilotes ; aucun programme n'y accède directement.

05

Exceptions

Traite les pannes matérielles et les erreurs de programme grâce aux interruptions.

Toutes ces tâches visent le même but : **partager et optimiser** l'utilisation des ressources entre les programmes.

Un bon OS est ergonomique, efficace et évolutif

Ergonomie

Offrir une vision **simple et uniforme** du matériel.

Ex. : enregistrer un fichier se fait de la même façon sur un disque dur ou une clé USB.

Efficacité

Donner un **accès rapide** à l'information et tirer le meilleur parti du processeur, de la mémoire et des disques.

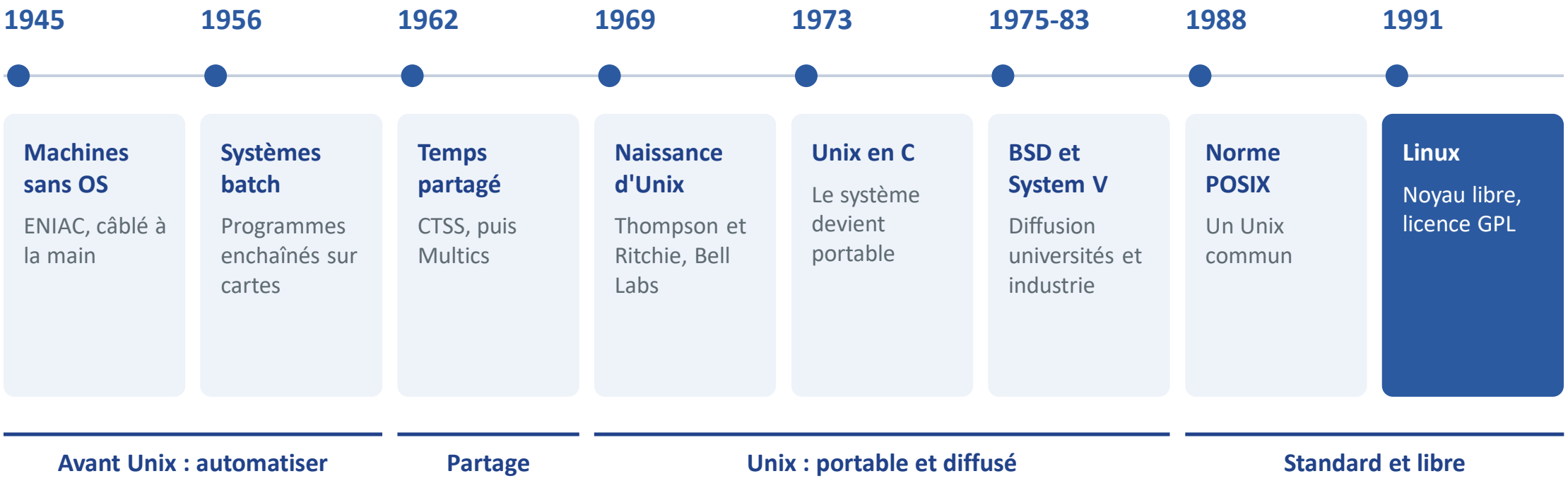
Ex. : le processeur passe à un autre programme pendant qu'un premier attend le disque.

Évolutivité

S'**adapter** aux nouveaux matériels et périphériques.

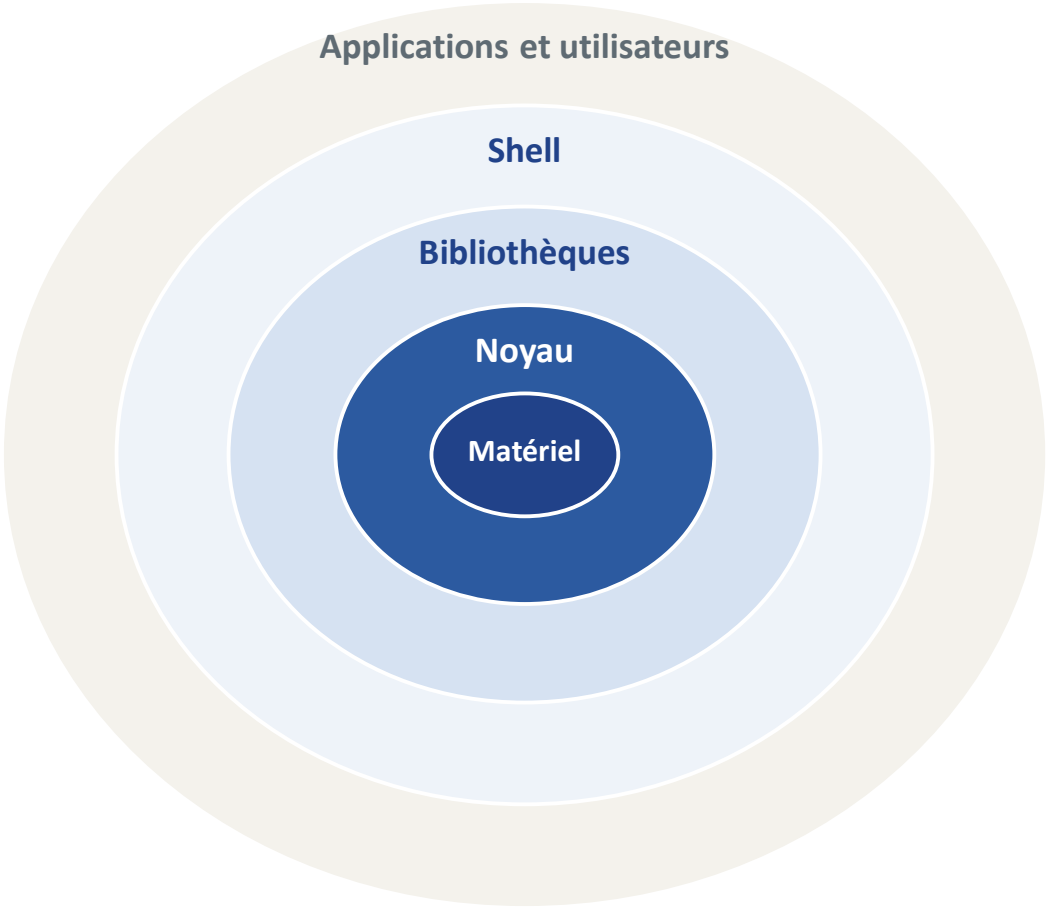
Ex. : brancher une nouvelle imprimante demande seulement d'installer son pilote.

Des premiers ordinateurs à Linux, chaque étape a rendu l'OS plus partagé et plus portable



Chaque étape répond à une limite de la précédente : attente, partage, portabilité, compatibilité.

Unix est organisé en couches autour du noyau



Noyau

Chargé en mémoire au démarrage, il dialogue avec le matériel. Il gère la mémoire, partage le processeur entre les tâches et protège les processus les uns des autres.

Bibliothèques

Bibliothèques standard d'**appels système** : les programmes passent par elles pour demander un service au noyau.

Shell

L'**interpréteur de commandes** : il lit ce que tape l'utilisateur et le fait exécuter. C'est aussi un langage de programmation (chapitre 6).

Unix est multi-utilisateur, multitâche, hiérarchisé et protégé

Multi-utilisateur

Plusieurs utilisateurs se connectent et travaillent en même temps, chacun dans son propre environnement.

Multitâche

Un utilisateur lance plusieurs travaux à la fois, au premier plan, en arrière-plan ou en différé (batch).

Système de fichiers hiérarchisé

Tous les fichiers sont rangés dans une arborescence unique, à partir de la racine /.

Mécanisme de protection

Identification par mot de passe. **root** (super-utilisateur) a tous les droits et crée les comptes ; un utilisateur normal n'a que les droits qu'il lui accorde.

Exercice : Le rôle de l'OS au quotidien

Contexte

Sur votre téléphone, vous regardez une vidéo pendant qu'une application télécharge un fichier. L'appareil photo, lui, refuse de s'ouvrir dans une application sans autorisation.

Travail demandé

Q1. Quel **rôle** de l'OS (traducteur, chef d'orchestre, agent de sécurité) intervient dans chaque situation ?

Q2. Quelles **tâches de gestion** sont sollicitées : processus, mémoire, fichiers, E/S, exceptions ?

Q3. Quelle **qualité** d'un bon OS l'exemple illustre-t-il ?

Rappel

3 rôles : traducteur, chef d'orchestre, agent de sécurité.

5 tâches : processus, mémoire, fichiers, entrées/sorties, exceptions.

3 qualités : ergonomie, efficacité, évolutivité.

Exercice : Le rôle de l'OS au quotidien

```
Vidéo + téléchargement en même temps
  -> chef d'orchestre
     (partage du processeur)

Lecture de la vidéo, écriture du
fichier sur le stockage
  -> traducteur

Caméra refusée sans permission
  -> agent de sécurité

Tâches : processus, mémoire,
fichiers, entrées/sorties
Qualité : efficacité
```

Explications

- Une même situation mobilise souvent plusieurs rôles.
- Partager le processeur entre programmes, c'est la **multiprogrammation**.
- La caméra est un périphérique : seul l'OS, via son pilote, y accède.

Chapitre 1 · Les commandes de base

Session et shell, fichiers et chemins, manipulation, droits, redirections, traitement et recherche

Pour travailler sous Unix, on ouvre une session puis on la referme

1 · Connexion

Unix est multi-utilisateur : chacun s'identifie par un **nom d'utilisateur** et un **mot de passe**.

Vous arrivez ensuite dans votre **répertoire de connexion** (home directory).

```
login : salwa
password : ******
$ pwd
/home/salwa
```

2 · Changer d'utilisateur

`su` prend l'identité d'un autre utilisateur ; son mot de passe est demandé.

`su [-] utilisateur [-c commande]`

`-` charge l'environnement du nouvel utilisateur ; `-c` exécute la commande indiquée.

```
$ su - root
Mot de passe :
$ exit  # retour au shell précédent
```

3 · Déconnexion

Pour fermer la session, tapez `exit` ou la combinaison de touches **Ctrl+D**.

Ctrl+D signale au shell la fin de la saisie : le shell se termine et la session se ferme.

Après un `su`, `exit` ramène d'abord à l'utilisateur précédent.

```
$ exit
```

Le shell interprète chaque commande tapée au prompt



Le **shell** est l'interface entre l'utilisateur et le noyau ; on l'appelle aussi **interpréteur de commandes**. Après chaque commande, il réaffiche le prompt.

Pour une commande externe, il crée un **nouveau processus** et attend sa fin. Ouvrir une console lance déjà un processus shell.

Shell	Chemin	Origine
sh	`/bin/sh`	Bourne shell, ancêtre des shells
bash	`/bin/bash`	Bourne Again shell, version améliorée
ksh	`/bin/ksh`	Korn shell, Bourne étendu par AT&T
csh	`/bin/csh`	C shell, développé à Berkeley (BSD)

```
# Prompt : nomSession@nomMachine:~$
$ echo $SHELL
/bin/bash
$ cat /etc/shells  # shells disponibles
/bin/sh
/bin/bash
/bin/ksh
```

Sous Unix, tout est fichier, et chaque fichier a un type

Un fichier est une **séquence d'octets** qui porte un nom et se range dans un répertoire.

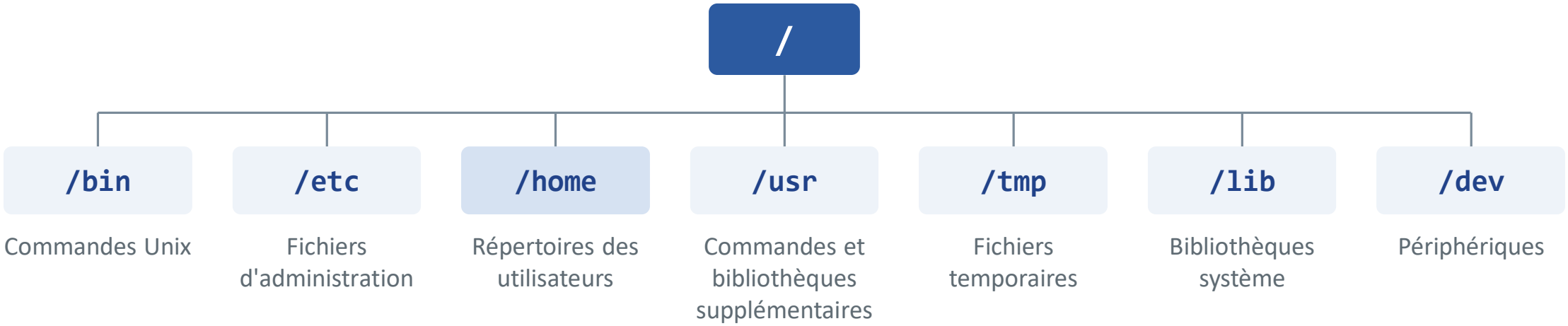
Le noyau n'impose **aucune structure** à son contenu.

Il n'y a pas de notion de disque physique : les **périphériques** eux-mêmes sont des fichiers.

Le type est le **premier caractère** affiché par `ls -l`.

Type	Lettre	Rôle
Fichier ordinaire	`-`	Données manipulées par les utilisateurs
Répertoire	`d`	Contient d'autres fichiers
Lien symbolique	`l`	Pointe vers un autre fichier
Tube nommé	`p`	Communication entre processus
Socket	`s`	Communication, y compris en réseau
Spécial caractère	`c`	Périphérique lu caractère par caractère (terminal)
Spécial bloc	`b`	Périphérique lu par blocs (disque)

Les fichiers forment un arbre unique dont la racine est /



Deux structures

Logique : l'arbre que voit l'utilisateur. Les nœuds sont des répertoires, les feuilles des fichiers.

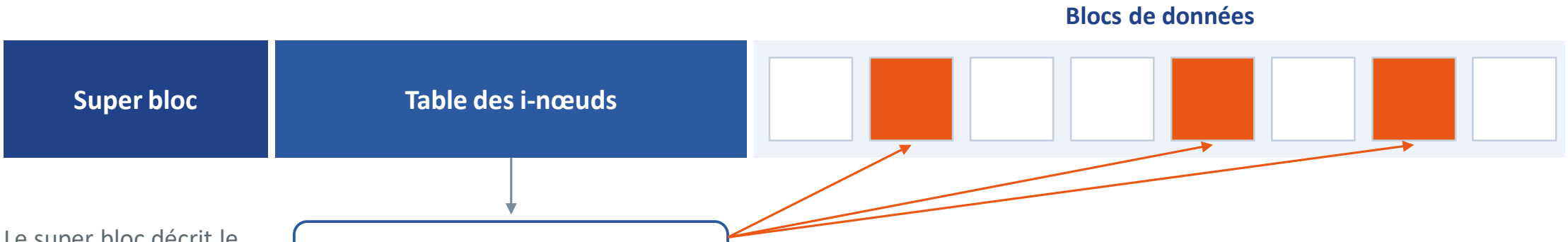
Physique : la façon dont les octets sont codés sur le disque.

Situer un fichier

Un fichier se repère par son **nom** et son **chemin d'accès**, ex. `/home/etudiant1/notes.txt`.

Le caractère `/` sépare les niveaux de l'arbre.

Chaque fichier est décrit par un i-nœud qui pointe vers ses blocs de données



Le super bloc décrit le système de fichiers et répertorie les **blocs libres**.

Un i-nœud

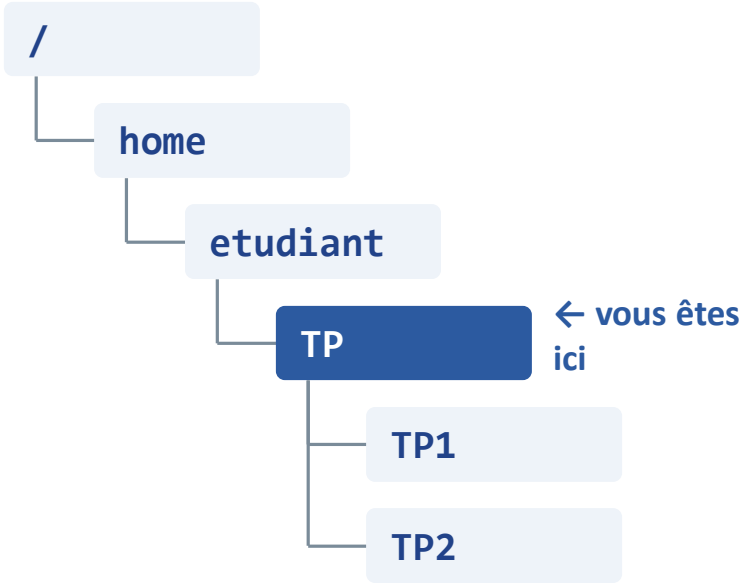
- Type du fichier
- Droits d'accès
- Propriétaire et groupe
- Taille
- Dates d'accès et de modification
- Nombre de liens
- **Adresses des blocs de données**

Le **nom** du fichier n'est pas dans l'i-nœud : un répertoire associe chaque nom à un **numéro d'i-nœud**.

`ls -i` affiche ce numéro.

Cette organisation physique est **transparente** pour l'utilisateur, qui ne voit que l'arbre.

Un chemin absolu part de la racine, un chemin relatif part du répertoire courant




```
$ pwd
/home/etudiant/TP
# pwd affiche toujours
# un chemin absolu
```

Chemin absolu

Il part de la **racine** `/` et désigne le même fichier, où que vous soyez.

Ex. : `/home/etudiant/TP/TP1`

Chemin relatif

Il part du **répertoire courant**. Depuis TP : `TP1` ou `./TP1`

`.` répertoire courant · `..` répertoire parent · `~` répertoire de connexion

Ex. : depuis TP, `../..` désigne `/home`

Le shell enchaîne les commandes et distingue commandes internes et externes

Enchaîner des commandes

Syntaxe	Effet
`cmd1 ; cmd2`	cmd2 s'exécute après cmd1, quel que soit son résultat
`cmd1 && cmd2`	cmd2 s'exécute seulement si cmd1 **réussit**
`cmd1 || cmd2`	cmd2 s'exécute seulement si cmd1 **échoue**

```
$ mkdir tp ; cd tp
$ mkdir tp && cd tp
$ cd tp || mkdir tp
```

Deux sortes de commandes

Interne : son code est intégré au shell. Ex. : `cd`, `echo`, `pwd`.

Aucun nouveau processus n'est créé.

Externe : son code est dans un fichier. Ex. : `ls`, `mkdir`, `sleep`, `vi`, scripts shell.

Le shell crée un **processus fils** à chaque exécution.

which, man et alias aident à trouver, comprendre et raccourcir les commandes

`which`

Où est une commande ?

```
$ which ls
/usr/bin/ls
$ which mkdir
/usr/bin/mkdir
```

Rien pour une commande interne comme cd : utilisez `type cd`.

`man`

Que fait une commande ?

```
$ man ls
# q pour quitter
# /motif pour chercher
$ ls --help
```

`--help` affiche un résumé rapide des options.

`alias`

Raccourcir une commande

```
$ alias lsl='ls -l'
$ alias lsd='ls -d'
$ alias        # les lister
$ unalias lsl
```

Pour garder un alias, ajoutez-le dans `~/.bashrc`.

Les métacaractères ont un sens spécial pour le shell

Caractère	Signification
*	Toute suite de caractères
?	Un seul caractère
[ ]	Un caractère d'une liste ou d'un intervalle
~	Répertoire de connexion
< > >>	Redirections
|	Tube (pipe)
$ ${…}	Valeur d'une variable

Caractère	Signification
;	Sépare des commandes
( ) { }	Groupent des commandes
&& ||	Associent des commandes
!	Négation logique
&	Lance en arrière-plan
#	Début de commentaire
' " \	Protègent les autres métacaractères

```
$ ls *.txt     # tous les fichiers .txt
$ ls fic?.c    # fic1.c, ficA.c… (un seul caractère après fic)
$ ls [a-c]*    # les noms qui commencent par a, b ou c
```

echo affiche ses arguments ; ' " et \ décident de ce que le shell interprète

`echo` écrit ses paramètres sur la sortie standard (l'écran). Exemple : `echo "Bonjour, ça va ?"`

' apostrophes

PROTECTION FORTE

Rien n'est interprété, pas même `$`.

Elles vont par paires : un nombre impair affiche le prompt secondaire `>`.

```
$ echo *
rep1 fic
$ echo '*'
*
$ a=6
$ echo '$a'
$a
```

" guillemets

PROTECTION FAIBLE

Tout est protégé sauf `$`, `\`, `"` et les apostrophes inverses : les **variables sont remplacées**.

```
$ a=6
$ echo "a vaut $a"
a vaut 6
$ echo "bonjour l'ami"
bonjour l'ami
```

\ antislash

PROTÈGE UN CARACTÈRE

Annule le sens spécial du **seul caractère qui suit**.

```
$ echo \$a
$a
$ echo \'
'
$ echo "Bonjour \\"
Bonjour \
```

touch crée un fichier vide, mkdir crée un répertoire

`touch`

Syntaxe : `touch fichier`

- Si le fichier **n'existe pas**, il est créé, vide.
- S'il **existe**, ses dates d'accès et de modification prennent la date et l'heure courantes.

```
$ touch test.log
$ ls -l test.log
-rw-r--r-- 1 salwa salwa 0
   oct.  5 10:12 test.log
```

`mkdir`

Syntaxe : `mkdir [options] répertoire`

Option	Effet
`-p`	Crée aussi les répertoires parents manquants
`-m`	Fixe les droits d'accès du répertoire
`-v`	Affiche chaque création

```
$ mkdir tp1
$ mkdir -p rep1/test/image
$ mkdir -v rep2
mkdir: création du répertoire 'rep2'
```

cd change le répertoire courant, avec un chemin absolu ou relatif

Syntaxe : `cd [chemin vers le répertoire]`

Commande	Effet
`cd rep`	Va dans rep (chemin absolu ou relatif)
`cd .`	Reste dans le répertoire courant
`cd ..`	Monte d'un niveau (répertoire parent)
`cd ../..`	Monte de deux niveaux
`cd ~` ou `cd`	Va dans le répertoire de connexion
`cd -`	Revient au répertoire précédent

Le répertoire de destination doit **exister**, sinon le shell affiche une erreur.

```
$ pwd
/home/etudiant/TP
$ cd ..           # vers le parent
$ pwd
/home/etudiant
$ cd TP/TP2       # chemin relatif
$ pwd
/home/etudiant/TP/TP2
$ cd /etc         # chemin absolu
$ cd -            # retour
/home/etudiant/TP/TP2
```

ls liste le contenu d'un répertoire, et ses options changent ce qu'il montre

Syntaxe : `ls [options] [chemin]` · sans chemin, `ls` liste le répertoire courant (équivaut à `ls .`)

Ce que ls affiche

Option	Effet
`-l`	Format long : type, droits, liens, taille, date
`-a`	Inclut les fichiers cachés
`-R`	Parcourt aussi les sous-répertoires
`-i`	Numéro d'i-nœud de chaque fichier
`-d`	Le répertoire lui-même, pas son contenu
`-F`	Suffixe de type : / répertoire, * exécutable, @ lien
`-p`	Ajoute / après les répertoires

Comment ls trie et présente

Option	Effet
`-t`	Trie par date : plus récents d'abord
`-r`	Inverse l'ordre (`-rt` : plus anciens d'abord)
`-s`	Taille en blocs ; `-sh` en Ko, Mo lisibles
`-u`	Date du dernier accès (avec -l ou -t)
`-c`	Date du dernier changement (avec -l ou -t)
`-m`	Noms séparés par des virgules
`-1 -C`	Une seule colonne / plusieurs colonnes

Les options se combinent : `ls -ltr`, `ls -la`. Pour un long listing page par page : `ls -l | less`.

Une ligne de ls -l se lit en huit champs




```
$ ls -l
drwxr-xr-x 2 salwa grp1 512 Mar 06 15:09 CVS
-rw-r--r-- 1 salwa grp1 744 Mar 06 14:17 Makefile
```

Droits : trois groupes de trois lettres, pour le propriétaire, le groupe et les autres (r lecture, w écriture, x exécution).

Nombre de liens : en général 1 pour un fichier ; pour un répertoire, 2 + le nombre de ses sous-répertoires.

Les **jokers** s'appliquent :

```
ls -l [a-r]?*.txt
```

liste les fichiers .txt dont le nom commence par une lettre de a à r et compte au moins deux caractères avant .txt.

mv déplace ou renomme fichiers et répertoires

Syntaxe : `mv [options] source destination`

Commande	Effet
`mv ~/rep1/stat24.log .`	Déplace stat24.log dans le répertoire courant
`mv file.txt rep1`	Déplace file.txt (du répertoire courant) dans rep1
`mv ancien.txt nouveau.txt`	Renomme le fichier
`mv rep1 repf`	Si repf existe, y déplace rep1 ; sinon, renomme rep1 en repf

Options utiles : `-i` demande confirmation avant d'écraser un fichier existant · `-v` affiche chaque déplacement.

rm supprime définitivement ; rmdir ne supprime que les répertoires vides

`rm`

Syntaxe : `rm [options] fichiers`

- `-i` confirmer chaque suppression
- `-v` afficher chaque suppression
- `-r` ou `-R` supprimer un répertoire et tout son contenu

```
$ rm notes.txt
$ rm *.txt      # tous les .txt
$ rm *          # tous les fichiers ici
$ rm -r rep     # rep et son contenu
```

`rmdir`

Syntaxe : `rmdir [options] répertoire`

- Supprime un répertoire **vide**.
- Si le répertoire contient des fichiers, rmdir refuse : utilisez `rm -r`.

```
$ rmdir tp1
$ rmdir rep1
rmdir: impossible de supprimer 'rep1':
  Le dossier n'est pas vide
```

Pas de corbeille : un fichier supprimé par rm est perdu. En cas de doute, surtout avec des jokers, ajoutez `-i`.

cp copie un fichier ou un répertoire, et la source reste intacte

Syntaxe : `cp [options] source destination`

Commande	Résultat
`cp log/stat24.log stat/stat25.log`	Copie sous un nouveau nom
`cp log/stat24.log stat/`	Copie sous le même nom dans stat
`cp ./dir1/cmd1.sh ./dir2/cmd2.sh`	dir2 contient cmd2.sh
`cp ./dir1/cmd1.sh ./dir2`	dir2 contient cmd1.sh
`cp -R rep1 rep2`	Copie rep1 avec tout son contenu

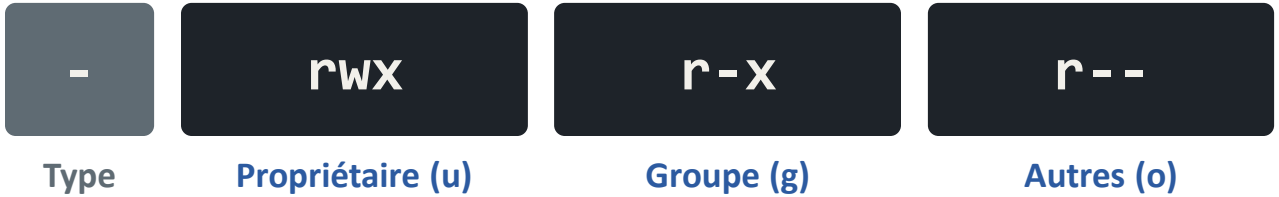
Sans `-R`, cp refuse de copier un répertoire.

Options utiles

- `-i` confirmer avant d'écraser
- `-R` ou `-r` copier un répertoire et son contenu
- `-v` afficher chaque copie

Attention : cp écrase sans prévenir une destination qui existe déjà. L'option `-i` demande confirmation avant.

Chaque fichier porte trois séries de droits : propriétaire, groupe et autres



Les 10 premiers caractères de `ls -l` : le type, puis trois groupes de trois lettres. Un tiret signifie que le droit est absent.

Droit	Fichier ordinaire	Répertoire
`r` **lecture**	Lire le contenu (`cat`)	Lister le contenu (`ls`)
`w` **écriture**	Modifier le contenu	Créer, supprimer, renommer des fichiers dedans (`touch`, `mkdir`, `rm`, `mv`, `cp`)
`x` **exécution**	Exécuter un programme ou un script	Traverser le répertoire (`cd`)

Pour supprimer un fichier, c'est le droit `w` sur le **répertoire** qui compte, pas sur le fichier.

chmod change les droits ; la notation symbolique ajoute, retire ou fixe des droits

Syntaxe : `chmod [ugoa][+-=][rwx] fichier` · seul le propriétaire du fichier (ou root) peut changer ses droits

Qui ?

- `u` propriétaire (user)
- `g` groupe
- `o` autres
- `a` tous (u+g+o)

Opération

- `+` ajouter
- `-` retirer
- `=` fixer exactement

Quels droits ?

- `r` lecture
- `w` écriture
- `x` exécution

```
$ chmod go+w file
# écriture pour le groupe et les autres

$ chmod a=rw file
# lecture et écriture pour tous

$ chmod go-rw file
# retire r et w au groupe et aux autres

$ chmod u+x,g-w+x,o=r file
# u : +x · g : -w +x · o : r seul
```

Sans lettre de classe (`chmod +x f`), toutes les classes sont concernées.

En notation octale, chaque série de droits devient un chiffre de 0 à 7

Binaire	Octal	Droits
000	0	`---`
001	1	`--x`
010	2	`-w-`
011	3	`-wx`
100	4	`r--`
101	5	`r-x`
110	6	`rw-`
111	7	`rwx`

Chaque droit vaut un bit : **r = 4**, **w = 2**, **x = 1**. On additionne les droits accordés.

Syntaxe : `chmod abc fichier`, avec a, b, c entre 0 et 7 pour le propriétaire, le groupe et les autres.

7
`4+2+1 → rwx`
propriétaire

5
`4+1 → r-x`
groupe

4
`4 → r--`
autres

```
$ chmod 754 script.sh
$ ls -l script.sh
-rwxr-xr-- 1 toto grp1 210 oct.  5 11:02 script.sh
```

umask fixe les droits retirés à chaque création de fichier ou de répertoire

On part des droits **maximum** :

- `666` (`rw-rw-rw-`) pour un fichier ordinaire
- `777` (`rwxrwxrwx`) pour un répertoire

puis on retire les droits indiqués par le masque `umask`.

	Fichier	Répertoire
Maximum	666	777
umask	022	022
Résultat	644 · `rw-r--r--`	755 · `rwxr-xr-x`

```
$ umask
0022
$ umask 0077
$ touch secret.txt
$ mkdir prive
$ ls -ld secret.txt prive
-rw------- 1 toto grp1 0 ... secret.txt
drwx------ 2 toto grp1 4096 ... prive
```

Le masque ne vaut que pour le shell courant. Pour le garder, ajoutez la commande `umask` dans `~/.bashrc`.

Chaque commande lit son entrée standard et écrit sur deux sorties



Trois flux, chacun avec un **descripteur** :

- `0` entrée standard, le clavier
- `1` sortie standard, l'écran
- `2` sortie d'erreur, l'écran

Rediriger, c'est remplacer le clavier ou l'écran par un fichier.

Opérateur	Effet
`cmd > f`	Envoie la sortie standard dans f ; f est créé ou **écrasé**
`cmd >> f`	Ajoute la sortie standard à la fin de f
`cmd 2> f` ou `2>> f`	Envoie les erreurs dans f (écrase / ajoute)
`cmd < f`	Lit l'entrée standard dans f au lieu du clavier
`cmd > f 2>&1` ou `cmd &> f`	Envoie la sortie et les erreurs dans le même fichier

> écrase, >> ajoute, 2> isole les erreurs et < fournit l'entrée

Sortie standard

```
$ ls -l > res.txt
# res.txt créé ou écrasé
$ date >> res.txt
# ajouté à la fin
```

Erreurs

```
$ eccho bonjour 2> err.txt
$ cat err.txt
eccho : commande introuvable
$ ls /inexistant 2>/dev/null
# erreurs jetées
```

Entrée standard

```
$ wc -l < villes.txt
3
$ sort < noms.txt > tries.txt
# lit noms.txt,
# écrit dans tries.txt
```

`/dev/null` est un fichier spécial qui absorbe tout ce qu'on y écrit : la « poubelle » du système.

Piège : `sort f > f` vide f avant que sort ne le lise. Écrivez toujours dans un autre fichier.

Le tube | branche la sortie d'une commande sur l'entrée de la suivante



Syntaxe : `cmd1 | cmd2`

Aucun fichier intermédiaire n'est créé.

Opérateur	Relie
`> <`	une commande à un **fichier**
`|`	une commande à une **autre commande**

On peut enchaîner autant de commandes que nécessaire : chaque commande filtre le résultat de la précédente.

Les commandes qui lisent l'entrée standard et écrivent sur la sortie standard s'appellent des **filtres** : cat, sort, wc, cut, head, tail, grep, tr.

```
$ echo "bonjour" | cat
bonjour
$ ls -l | less       # défiler page à page
$ ls | wc -l         # nombre de fichiers
12
```

cat affiche et concatène des fichiers ; less les fait défiler page par page

`cat`

Commande	Effet
`cat f`	Affiche f à l'écran
`cat f1 f2`	Affiche f1 puis f2 (concaténation)
`cat f1 f2 > f3`	Regroupe f1 et f2 dans f3
`cat -n f1 f2`	Concatène et numérote les lignes
`cat > f`	Saisit du texte au clavier jusqu'à Ctrl+D

cat est d'abord une commande de **concaténation** vers la sortie standard.

`less`

Pour un long fichier : `less f`, ou `cat f | less`. Le texte s'affiche écran par écran.

Touche	Action
Espace	Page suivante
b	Page précédente
/motif	Rechercher motif
q	Quitter

```
$ cat -n villes.txt | less
```

wc compte les lignes, les mots et les caractères d'un fichier

Syntaxe : `wc [options] fichier`

Option	Compte
`-l`	Les lignes
`-w`	Les mots
`-c`	Les octets (caractères en ASCII)
`-m`	Les caractères, accents compris
`-L`	La longueur de la ligne la plus longue

Sans option, wc affiche lignes, mots et octets, dans cet ordre.

Avec un tube, wc compte le résultat d'une autre commande.

```
$ cat villes.txt
Agadir:Maroc:80000
Rabat:Maroc:10000
Tanger:Maroc:94700
$ wc villes.txt
3  3 56 villes.txt
$ wc -l villes.txt
3 villes.txt
$ wc -L villes.txt
18 villes.txt
$ ls | wc -l   # fichiers du répertoire
```

cut extrait des caractères ou des champs de chaque ligne

Par caractères · `-c`

Commande	Extrait, sur chaque ligne
`cut -c3 f`	le 3e caractère
`cut -c3- f`	du 3e caractère à la fin
`cut -c-3 f`	les 3 premiers caractères
`cut -c1,5 f`	les caractères 1 et 5
`cut -c2-5 f`	les caractères 2 à 5

Par champs · `-d` séparateur, `-f` champs

Commande	Extrait, sur chaque ligne
`cut -d: -f2 f`	le 2e champ
`cut -d: -f2- f`	du 2e champ à la fin
`cut -d: -f-2 f`	les 2 premiers champs
`cut -d: -f1,3 f`	les champs 1 et 3
`cut -d: -f1-3 f`	les champs 1 à 3

```
$ cut -d: -f1 villes.txt
Agadir
Rabat
Tanger
```

```
$ cut -d: -f1,3 villes.txt
Agadir:80000
Rabat:10000
Tanger:94700
```

Le séparateur par défaut est la **tabulation**. Pour l'espace : `-d' '`.
Autres séparateurs courants : `, ; :`

head affiche le début d'un fichier, tail en affiche la fin

Commande	Affiche
`head f`	Les 10 premières lignes
`head -n 20 f`	Les 20 premières lignes
`tail f`	Les 10 dernières lignes
`tail -n 20 f`	Les 20 dernières lignes
`tail -n +20 f`	De la 20e ligne jusqu'à la fin
`tail -f journal.log`	La fin, puis les lignes ajoutées en direct (Ctrl+C pour arrêter)

Sans `-n`, head et tail affichent **10 lignes**.

Combiner les deux

Pour une tranche du fichier, on coupe le début avec head puis on garde la fin avec tail.

```
# lignes 11 à 20
$ head -n 20 f | tail -n 11

# dernière ligne
$ tail -n 1 f
```

sort trie les lignes d'un fichier, par défaut dans l'ordre alphabétique

Option	Effet
`-n`	Tri numérique : 12 avant 100
`-r`	Ordre décroissant
`-k2,2`	Trier sur le 2e champ seulement
`-t:`	Séparateur de champs « : »
`-f` / `-b`	Ignorer la casse / les espaces en tête

Sans `-n`, les nombres sont triés comme du texte : « 100 » passe avant « 12 ».

```
$ cat notes.txt
1 fatima 36
1 ahmad 30
3 salma 78
6 bouchra 100
2 ahmad 78
4 bouchra 90
9 amine 12
```

```
$ sort -k2,2 notes.txt
1 ahmad 30
2 ahmad 78
9 amine 12
4 bouchra 90
6 bouchra 100
1 fatima 36
3 salma 78
```

```
$ sort -n -k3,3 notes.txt
9 amine 12
1 ahmad 30
1 fatima 36
2 ahmad 78
3 salma 78
4 bouchra 90
6 bouchra 100
```

split découpe un fichier en morceaux ; basename et dirname séparent nom et chemin

split

Syntaxe : `split -l n fichier préfixe`

Crée préfixeaa, préfixeab… de n lignes chacun.

```
$ wc -l journal.txt
25 journal.txt
$ split -l 10 journal.txt part_
$ ls part_*
part_aa  part_ab  part_ac
# 10, 10 et 5 lignes
$ cat part_* > copie.txt   # recoller
```

basename · dirname

`basename` garde le **nom** du fichier ; `dirname` garde le **chemin** du répertoire qui le contient.

```
$ basename /home/TP/liste.log
liste.log
$ basename /home/TP/liste.log .log
liste
$ dirname /home/TP/liste.log
/home/TP
```

Très utiles dans les scripts, pour renommer ou ranger des fichiers.

find parcourt une arborescence et sélectionne les fichiers selon des critères

Syntaxe : `find [où] [critères] [action]` · find explore le répertoire indiqué et tous ses sous-répertoires

Critère	Sélectionne
`-name motif`	par nom (mettre le motif entre guillemets)
`-type f`	par type : f, d, l, b, c, s
`-user` / `-group`	par propriétaire / groupe
`-size +10M`	par taille : + plus de, - moins de (k, M, G)
`-mtime -5`	par date de modification, en jours
`-perm 644`	par droits d'accès
`-maxdepth 1`	limite la profondeur de recherche

```
$ find . -name "*test"     # finit par test
$ find . -name "*test*"    # contient test
$ find . -type d -name "Unix*"
$ find TP/TP1 -type f -name "Unix*"
$ find . ! -user ali     # pas à ali
$ find . -size +10M
$ find . -mtime -5         # moins de 5 j
```

`-mtime 5` : il y a 5 jours · `+5` : plus de 5 jours · `-5` : moins de 5 jours

find combine ses critères avec -a, -o et !, puis agit sur les résultats avec -exec ou -ok

Combiner les critères

Opérateur	Sens
(rien) ou `-a`	ET : les deux critères sont vrais
`-o`	OU : l'un des critères est vrai
`!`	NON : le critère est faux
`\( … \)`	Groupe des critères

```
$ find . -name "*unix*" -type f
$ find . -type f \( -name "*.txt" \
    -o -name "*.md" \)
$ find . \( -size +10M -o -size 10M \) \
    -a -name "*unix*"
```

Agir sur chaque fichier trouvé

- `-exec commande {} \;` exécute la commande sur chaque résultat ; `{}` est remplacé par le nom du fichier.
- `;` termine la commande : il est protégé par `\`.
- `-ok` fait la même chose mais **demande confirmation**.

```
$ find . -name "*.c" -exec rm {} \;
$ find . -name "Fichier1" -ok rm {} \;
< rm ... ./Fichier1 > ? y
```

`-exec rm` supprime **sans confirmation** : testez d'abord sans -exec.

grep affiche les lignes d'un fichier qui contiennent un motif

Syntaxe : `grep [options] motif [fichiers]` · sans fichier, grep lit l'entrée standard (utile après un tube)

Option	Effet
`-c`	Affiche le nombre de lignes trouvées
`-i`	Ignore majuscules et minuscules
`-l`	Liste les fichiers qui contiennent au moins une ligne trouvée
`-n`	Affiche le numéro de chaque ligne trouvée
`-v`	Affiche les lignes qui ne contiennent pas le motif
`-r`	Cherche dans tout un répertoire

```
$ grep "etudiant" /etc/passwd
etudiant:x:1000:1000::/home/etudiant:/bin/bash
$ grep -c "Maroc" villes.txt
3
$ grep -n "Rabat" villes.txt
2:Rabat:Maroc:10000
$ grep -i "tanger" villes.txt
Tanger:Maroc:94700
```

Les expressions régulières décrivent des motifs ; egrep (grep -E) accepte leur forme étendue

Expressions de base (grep)

Motif	Correspond à
`.`	un caractère quelconque
`*`	l'élément précédent, 0 fois ou plus
`[abc]` / `[a-z]`	un caractère de la liste
`[^0-9]`	un caractère hors de la liste
`^` / `$`	le début / la fin de la ligne
`\{n\}`	n répétitions de l'élément précédent

Expressions étendues (egrep, grep -E)

Motif	Correspond à
`?`	l'élément précédent, 0 ou 1 fois
`+`	l'élément précédent, 1 fois ou plus
`{n}`	n répétitions, sans antislash
`a|b`	a ou b
`( )`	délimite un groupe

```
$ grep "^[0-9]\{4\}$" f    # lignes de 4 chiffres exactement
$ grep "^[A-Z]" villes.txt  # commencent par une majuscule
$ egrep "c[a-z]?t" f       # ct, cat, cot…
$ egrep "Rabat|Tanger" villes.txt
```

Ne confondez pas avec les jokers du shell : ici * répète l'élément précédent. Mettez le motif entre guillemets.

tr remplace des caractères ; diff compare deux fichiers ligne par ligne

tr

Syntaxe : `tr [options] source cible`

tr lit **uniquement l'entrée standard** : on lui donne un fichier par `<` ou par un tube.

```
$ cat prix.txt
Produit prix quantites
$ cat prix.txt | tr 'oi' 'ea'
Preduat prax quantates
$ tr 'a-z' 'A-Z' < villes.txt
$ tr -d ':' < villes.txt   # supprime
```

Chaque caractère de la source est remplacé par celui de même rang dans la cible : o→e, i→a.

diff

Syntaxe : `diff [options] source cible`

```
$ diff v1.txt v2.txt
2c2
< Rabat
---
> Fès
```

Option	Effet
`-b` / `-B`	Ignorer espaces / lignes vides en plus
`-i`	Ignorer majuscules et minuscules
`-q` / `-s`	Dire seulement si différents / identiques
`-r`	Comparer deux répertoires récursivement

Synthèse (1/2) · session, fichiers et répertoires

Commande	Rôle	Exemple
`su`	Prendre l'identité d'un autre utilisateur	`su - root`
`exit`	Quitter le shell ou la session (ou Ctrl+D)	`exit`
`pwd`	Afficher le répertoire courant (chemin absolu)	`pwd`
`cd`	Changer de répertoire	`cd ../TP/TP2`
`ls`	Lister le contenu d'un répertoire	`ls -ltr`
`echo`	Afficher du texte ou une variable	`echo "a vaut $a"`
`touch`	Créer un fichier vide ou mettre à jour ses dates	`touch test.log`
`mkdir`	Créer un répertoire	`mkdir -p rep1/test`
`mv`	Déplacer ou renommer	`mv ancien.txt nouveau.txt`
`rm`	Supprimer des fichiers (et répertoires avec -r)	`rm -i *.txt`
`rmdir`	Supprimer un répertoire vide	`rmdir tp1`

Synthèse (2/2) · copies, droits, flux, traitement et recherche

Commande	Rôle
`cp`	Copier fichiers et répertoires (-R)
`ln` · `ln -s`	Créer un lien physique / symbolique
`chmod`	Changer les droits d'accès
`umask`	Fixer les droits par défaut
`> >> 2> <`	Rediriger les flux
`|`	Relier deux commandes
`cat` · `less`	Afficher, concaténer, faire défiler
`wc`	Compter lignes, mots, caractères

Commande	Rôle
`cut`	Extraire caractères ou champs
`head` · `tail`	Afficher le début / la fin
`sort`	Trier les lignes
`tr` · `diff`	Remplacer des caractères / comparer
`split`	Découper un fichier
`find`	Rechercher des fichiers
`grep` · `egrep`	Rechercher un motif dans un fichier
`which` · `man` · `alias`	Localiser, documenter, raccourcir

EXERCICES

Exercice 1 : Se déplacer dans l'arborescence

Contexte

Vous êtes connecté en tant que etudiant. Le répertoire courant est /home/etudiant/TP/TP1. L'arborescence contient aussi /home/etudiant/cours et /etc.

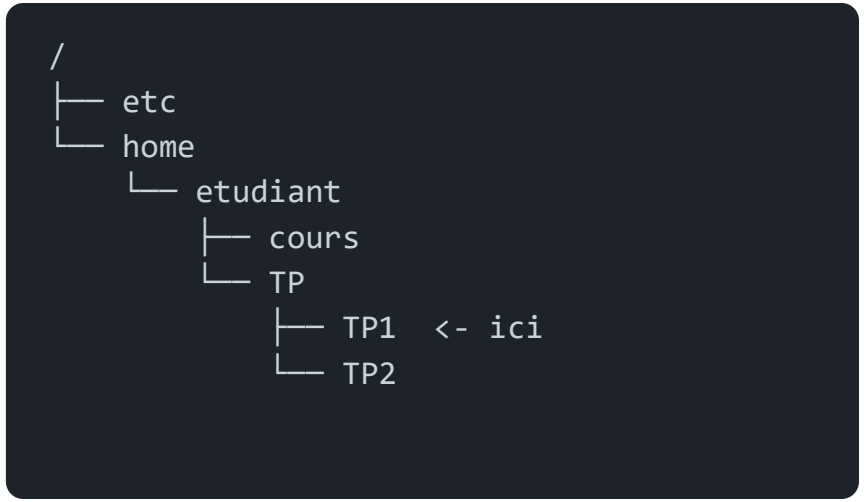
Travail demandé

Q1. Allez dans **TP2** (frère de TP1) avec un chemin **relatif**, puis **absolu**.

Q2. Allez dans **cours** avec le chemin relatif le plus court.

Q3. Revenez à votre **répertoire de connexion** de deux façons.

Q4. Que fait `cd -` juste après la question précédente ?



Corrigé · Exercice 1 · Se déplacer dans l'arborescence

```
# Q1
$ cd ../TP2
$ cd /home/etudiant/TP/TP2
# Q2 (depuis TP2)
$ cd ../../cours
# Q3
$ cd
$ cd ~
# Q4
$ cd -
/home/etudiant/cours
```

Explications

- `..` remonte d'un niveau : depuis TP2, `../..` mène à /home/etudiant.
- `cd` seul et `cd ~` vont au répertoire de connexion.
- `cd -` revient au répertoire précédent et l'affiche.

Exercice 2 · Chemins absolus et relatifs

Contexte

On considère l'arborescence ci-contre. L'utilisateur ali vient de se connecter : son répertoire courant est /home/ali.

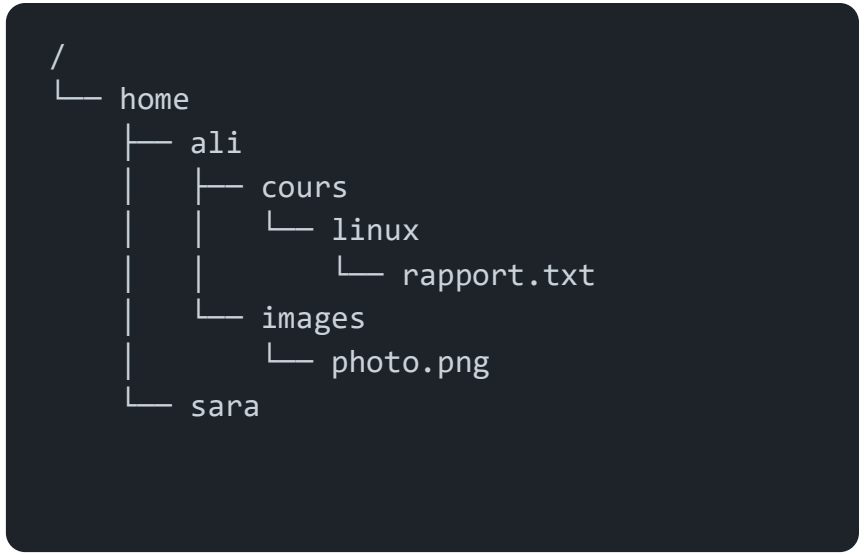
Travail demandé

Q1. Donnez le **chemin absolu** de rapport.txt.

Q2. Depuis /home/ali, donnez le **chemin relatif** de photo.png.

Q3. Vous êtes dans /home/ali/cours/linux. Écrivez la commande `cd` qui mène dans **images**, en relatif.

Q4. Après `cd ../../..`, que répond `pwd` ? Justifiez.



Corrigé · Exercice 2 · Chemins absolus et relatifs

```
# Q1
/home/ali/cours/linux/rapport.txt

# Q2 (depuis /home/ali)
images/photo.png      # ou ./images/photo.png

# Q3 (depuis /home/ali/cours/linux)
$ cd ../../images

# Q4
$ cd ../../..
$ pwd
/home
```

Explications

- Un chemin **absolu** commence toujours par `/` : il est valable partout.
- Chaque `..` remonte d'un niveau : linux → cours → ali → home.
- `pwd` affiche toujours un chemin absolu.

Exercice 3 · Créer une arborescence de travail

Contexte

Dans votre répertoire de connexion, vous devez créer l'arborescence ci-contre avec le moins de commandes possible.

Travail demandé

Q1. Créez **tous les répertoires** en une seule commande.

Q2. Créez les trois **fichiers vides** en une seule commande.

Q3. Que se passe-t-il si on tape `mkdir projet/src` alors que projet **n'existe pas** ? Comment l'éviter ?

Q4. Quelle commande affiche **toute l'arborescence** créée, sous-répertoires compris ?

```
~/projet
├── src
│   ├── main.c
│   └── util.c
├── doc
│   └── notes.txt
└── tests
```

Corrigé · Exercice 3 · Créer une arborescence de travail

```
# Q1
$ mkdir -p ~/projet/src ~/projet/doc ~/projet/tests
# ou : mkdir -p ~/projet/{src,doc,tests}

# Q2
$ cd ~/projet
$ touch src/main.c src/util.c doc/notes.txt

# Q3
$ mkdir projet/src
mkdir: impossible de créer 'projet/src':
  Aucun fichier ou dossier de ce type
# -> utiliser mkdir -p

# Q4
$ ls -R ~/projet
```

Explications

- `-p` crée les répertoires parents manquants et ne signale pas d'erreur s'ils existent.
- `touch` crée un fichier vide s'il n'existe pas.
- `ls -R` parcourt récursivement les sous-répertoires.

Exercice 4 · Copier, déplacer, renommer et supprimer

Contexte

Le répertoire courant contient a.txt, b.txt et le répertoire vide arch. On exécute dans l'ordre les commandes ci-contre.

Travail demandé

Q1. Pour chaque commande, décrivez son **effet**.

Q2. Donnez le **contenu final** du répertoire courant et de arch.

Q3. Pourquoi la commande 5 **échoue**-t-elle ? Corrigez-la.

Q4. Quelle option ajouter à rm pour **demander confirmation** avant chaque suppression ?

```
1  $ cp a.txt arch/
2  $ mv b.txt c.txt
3  $ mv c.txt arch/
4  $ cp -r arch sauv
5  $ rmdir arch
6  $ rm a.txt
```

Corrigé · Exercice 4 · Copier, déplacer, renommer et supprimer

```
1  copie a.txt dans arch (a.txt reste)
2  renomme b.txt en c.txt
3  déplace c.txt dans arch
4  copie arch et son contenu dans sauv
5  échec : arch n'est pas vide
6  supprime a.txt

Final : ./  -> arch/  sauv/
        arch/ et sauv/ -> a.txt  c.txt

# Q3 correction
$ rm -r arch

# Q4
$ rm -i fichier
```

Explications

- `mv` renomme si la destination n'existe pas, déplace si c'est un répertoire.
- `cp -r` est nécessaire pour copier un répertoire.
- `rmdir` ne supprime que les répertoires **vides** ; rm n'a pas de corbeille.